Ders 19 Çalışma Özeti

Ders 19 Çalışma Özeti

Genel Konular

- Graf (graph) veri yapısının günlük hayattaki kullanımları: harita uygulamaları, sosyal ağlar, internet ağı.
- Graf terminolojisi: düğüm (vertex / node), kenar (edge), derece (degree), yol (path), döngü (cycle).
- Bağlı bileşen (connected component), bağlı (connected) ve bağlı olmayan (disconnected) graf.
- Ağaç (tree) olarak graf ve iki parçalı graf (bipartite graph).
- Ağırlıklı graf (edge weighted graph), yönsüz (undirected) ve yönlü (directed) graf.
- Sembol tablosu (symbol table): düğüm adlarını dizi indislerine çevirme.
- Grafın bellekte gösterimi: komşuluk matrisi (adjacency matrix) ve komşuluk listesi (adjacency list).
- İki gösterimin yer ve zaman karmaşıklığı karşılaştırması. Seyrek matris (sparse matrix) durumu.
- Yönlü grafta giriş derecesi (in degree) ve çıkış derecesi (out degree).
- Kapsayan ağaç (spanning tree) ve en küçük kapsayan ağaç (minimum spanning tree, MST).
- Prim algoritması ve Kruskal algoritması. Kruskal için etiket dizisi tasarımı.
- Komşuluk listesinin C ile gerçekleştirilmesi: `struct node`, `struct Graph`, `createGraph`, `createNode`, `addEdge`.

Hocanın Özellikle Vurguladığı Kısımlar

- Graf, günümüzde gerçek dünya uygulamalarında en çok kullanılan veri yapılarından biridir. Ağ (network) ile gösterilebilen her problemde kullanılır.
- Terminolojiyi ezberlemek gerekmez. Ama gerektiği yerde doğru kullanmak önemlidir.
- Bilgisayar düğümleri adla değil, sayıyla (dizi indisiyle) işler. Kullanıcıya ad gösterilir, arka planda sayı kullanılır. Bunun için sembol tablosu gerekir.
- Matris mi liste mi kullanılacağı, tasarlanan algoritmanın ihtiyacına göre seçilir.
  - "İki düğüm bağlı mı?" sorusu çok soruluyorsa matris avantajlıdır. Doğrudan erişim vardır.
  - Düğüm sayısı çok büyük ama her düğüm az sayıda düğüme bağlıysa (seyrek graf) liste kullanılmalıdır.
- Yeni komşu düğüm listenin sonuna değil, başına eklenir. Sona kadar ilerlemek gereksiz zaman kaybıdır.
- Yönsüz grafta bir kenar eklenirken iki yönde de bağlantı kurulur: kaynak listesine hedef, hedef listesine kaynak eklenir.
- Kruskal'da iki düğümün etiketi aynıysa o kenar alınmaz. Alınırsa döngü oluşur.
- n düğümlü grafta $n - 1$ kenar alındığında en küçük kapsayan ağaç tamamlanmıştır.
- Hoca, Kruskal tasarımını ve `createNode` / `addEdge` kodlarını öğrencilerin kendi başlarına yazmasını istedi. Kendiniz yazmazsanız kalıcı olmaz.
- Kruskal uygulamasında girdideki kenarları sıralı kabul edebilirsiniz. Sıralama için hazır bir fonksiyon çağırmak yeterlidir.

Kısa Tekrar Notları

- Düğüm (vertex, çoğulu vertices) ile node aynı anlamdadır. Ağ uygulamalarında daha çok node denir.
- Kenar (edge): iki düğüm arasındaki bağlantı. Değeri (ağırlığı) olabilir: mesafe, süre, maliyet. Ağırlık bir sıfat da olabilir (iş arkadaşı, okul arkadaşı gibi).
- Derece: bir düğümün kaç düğüme bağlı olduğu.
- Yol (path): bir düğümden başka bir düğüme giderken izlenen kenarlar. Uzunluğu, kenar sayısıdır.
- Döngü (cycle): bir düğümden çıkıp aynı düğüme geri dönülebiliyorsa döngü vardır.
- Bağlı bileşen: grafın içinde kendi aralarında bağlı olan alt graf.
- Ağaç: yönsüz ve döngü içermeyen (acyclic) graf.
- İki parçalı graf: düğümler iki gruba ayrılır. Aynı grubun düğümleri kendi aralarında bağlanmaz. Örnek: öğrenciler ve aldıkları dersler, seyirciler ve izledikleri filmler.
- Yönsüz grafta A, B'ye bağlıysa B de A'ya bağlıdır. Yönlü grafta bu geçerli değildir (tek yönlü yol, Instagram'da takip).
- Komşuluk matrisi: bağlantı varsa 1, yoksa 0. Ağırlıklı grafta 1 yerine ağırlık yazılır. Yönsüz grafta matris simetriktir.
- Yönlü grafın matrisinde satır başlangıç düğümünü, sütun hedef düğümü gösterir.
- Komşuluk listesi: her düğüm için o düğümün komşularını tutan bir bağlı liste. Bu listeler bir dizide tutulur.
- Giriş derecesi: düğüme gelen kenar sayısı. Çıkış derecesi: düğümden çıkan kenar sayısı.
- Kapsayan ağaç: grafın tüm düğümlerini içeren ağaç.
- En küçük kapsayan ağaç: kenar ağırlıkları toplamı en küçük olan kapsayan ağaç.
- Derste çözülen 7 düğümlü örnekte Prim ve Kruskal aynı ağacı ve aynı toplamı (39) verdi.

İşlem	Komşuluk matrisi	Komşuluk listesi
Yer	V^2	$V + E$
v ile w bağlı mı?	$O(1)$	En kötü durumda listedeki eleman sayısı kadar (doğrusal arama)
v'nin herhangi bir komşusu var mı?	$O(V)$ (satırın tamamı)	$O(1)$ (liste başı `NULL` mı?)
v'nin tüm komşuları	Satırın tamamı taranır	Yalnız komşular gezilir

Detaylı Açıklamalar

Graf Nedir, Nerede Kullanılır?

- Graf, düğümlerden ve bu düğümler arasındaki kenarlardan oluşur.
- Harita uygulamalarında her bağlantı noktası bir düğümdür. Yollar kenarlardır. Kenarlarda süre, mesafe veya ulaşım bilgisi tutulabilir.
- Sosyal ağlarda kişiler düğümdür. Arkadaşlık veya takip ilişkileri kenardır.
- İnternetin kendisi de büyük bir graftır.
- Graf terminolojisi Ayrık Matematik (Discrete Mathematics) dersinde görüldüğü için hoca bu kısmı hatırlatma olarak hızlı geçti.

Temel Kavramlar

- Derece örneği (slayttaki graf): bir düğüm yalnız bir düğüme bağlıysa derecesi 1'dir. Dört düğüme bağlıysa derecesi 4'tür.
- Yol örneği: $A \to B \to C \to D \to E$ yolunda 4 kenar vardır. Bu yolun uzunluğu 4'tür.
- Döngü örneği:
  - Dört düğümlü, döngüsüz bir grafta hiçbir düğümden çıkıp aynı düğüme dönülemez.
  - Hoca bir kenar daha ekledi. Artık B düğümünden çıkıp C üzerinden yeniden B'ye dönülebiliyordu. Graf döngü içerir hâle geldi.
- Bağlı bileşen örneği:
  - Slayttaki graf birbirinden kopuk üç parçadan oluşuyordu. Her parça bir bağlı bileşendir.

  - Başka bir slayttaki bağlı olmayan graf dört bağlı bileşenden oluşuyordu.
- Ağaç ile graf ilişkisi: ağaç da bir graftır. Yönsüzdür ve döngü içermez. Bir kenar daha eklenip döngü oluşursa artık ağaç değildir.

Ağırlıklı, Yönsüz ve Yönlü Graf

- Kenar yalnız bağlantıyı gösterebilir. Örneğin iki kişinin sosyal ağda arkadaş olması.
- Kenarın bir değeri de olabilir. Bu durumda graf ağırlıklıdır (edge weighted).
- Yönsüz grafta yönün önemi yoktur. A'dan B'ye gidilebiliyorsa B'den A'ya da gidilebilir.
- Yönlü grafta kenarın yönü vardır.
  - Tek yönlü bir caddede bir yöne gidilir, geri dönüş başka yoldan yapılır.
  - Instagram'da takip ettiğiniz kişi sizi takip etmek zorunda değildir.
  - Web sayfalarındaki bağlantılar (hyperlink) da yönlü bir graftır.
- Öğrenci sorusu: Yönlü grafta $A \to D$ ve $D \to A$ kenarlarının ağırlıkları farklı olabilir mi?
  - Cevap: Evet. Örneğin iki semt arasında gidiş yolu 3 km, dönüş yolu 4 km olabilir.
  - Yönlü bir graf üzerinde çalışırken bu farka dikkat edilmelidir.
  - Bir düğüm kendine de bağlanabilir (self loop).

Sembol Tablosu

- Kullanıcı düğümleri adla görür: semt adı, cadde adı, havaalanı kodu.
- Bilgisayar ise grafı dizi veya matrisle saklar. Dizide "Esenler'inci göz" diye bir şey yoktur, sıfırıncı göz, birinci göz vardır.
- Bu yüzden iki yönlü bir sembol tablosu gerekir: ad $\leftrightarrow$ indis.
- Slayttaki havaalanı örneğinde 0 indisi bir havaalanı kodunu, 1 indisi başka bir kodu gösteriyordu.
- Tablonun hangi sırayla kurulacağı (alfabetik, haritadaki sıra vb.) tasarıma bağlıdır.

Komşuluk Matrisi

- V düğümlü graf için $V \times V$ boyutlu bir matris kullanılır.
- Ağırlıksız yönsüz grafta iki düğüm bağlıysa ilgili göze 1, değilse 0 yazılır.
- Yönsüz grafta 0, 1'e bağlıysa hem `[0][1]` hem `[1][0]` gözü 1 olur. Matris simetriktir.
- Ağırlıklı grafta 1 yerine kenarın ağırlığı yazılır. Slayttaki örnekte 0–1 kenarının ağırlığı 2'ydi. `[0][1]` ve `[1][0]` gözlerine 2 yazıldı.
- Yönlü grafta yalnız kenarın yönündeki göz dolar. Örnek: 5'ten 3'e kenar varsa `[5][3]` = 1, `[3][5]` = 0 olur.

Seyrek Graf ve Komşuluk Listesi

- Hocanın tavsiye sistemi örneği:
  - Milyonlarca kullanıcı ve on binlerce film olsun. Kullanıcı–film matrisi çok büyük olur.
  - Ama bir kullanıcı en fazla birkaç on veya yüz film izler. Her satırın büyük kısmı boştur.
  - Bu bir seyrek matristir. Matrisle saklamak çok fazla yer kaybettirir.
- Aynı durum sosyal ağlar için de geçerlidir. Milyarlarca kullanıcı vardır ama bir kişi yalnız birkaç yüz kişiye bağlıdır.
- 10 veya 1000 düğümlü küçük graflarda matris kullanmak sorun değildir.
- Düğüm sayısı çok büyük ve bağlantılar az ise yer karmaşıklığı (space complexity) düşünülerek liste seçilmelidir.
- Komşuluk listesinde her düğümün komşu sayısı farklıdır. Bu yüzden her düğümün komşuları bir bağlı listede tutulur.
- Dezavantaj: "0 ile 3 bağlı mı?" sorusu için 0'ın listesinde doğrusal arama (linear search) yapmak gerekir. Yerden kazanılır, zamandan kaybedilir.
- Ağırlıklı grafta liste düğümü üç parçalı olur: komşu düğüm numarası, kenar ağırlığı ve bir sonraki düğümün adresi.

Yer ve Zaman Karşılaştırması

- Matris için V^2 yer gerekir.
- Liste için $V + E$ yer gerekir. Yönsüz grafta her kenar iki listede görünür (0–1 ve 1–0).

  - Derste 3 düğüm ve 2 kenarlı örnekte listelerde 4 bağlantı kaydı vardı.
- Bağlantı sorgusu: matriste tek göze bakılır, $O(1)$. Listede en kötü durumda listenin tamamı taranır.
- "Düğümün herhangi bir komşusu var mı?" sorusu: matriste satırın tamamı taranır, $O(V)$. Listede liste başının `NULL` olup olmadığına bakılır, $O(1)$.
- "Tüm komşuları ver" sorusu: matriste satır taranır. Listede yalnız komşular gezilir.

Yönlü Grafta Giriş ve Çıkış Derecesi

- Yönsüz grafta yalnız derece vardır.
- Yönlü grafta giriş derecesi (gelen kenar sayısı) ve çıkış derecesi (giden kenar sayısı) ayrı ayrı tanımlanır.
- Slayttaki örnek: 6 numaralı düğüme yalnız 7'den gelinir, giriş derecesi 1'dir. 6'dan 0, 4 ve 9'a gidilir, çıkış derecesi 3'tür.
- Yönlü grafın komşuluk listesinde bir düğümün listesinde yalnız o düğümden gidilebilen düğümler bulunur.

En Küçük Kapsayan Ağaç

- Problem: Bir ağdaki tüm cihazlar birbirine bağlı olsun. Bağlantı (örneğin fiber optik kablo) en az maliyetle kurulsun.
- Üç şart vardır:
  - Tüm düğümler ağaçta yer alır (kapsayan).
  - Döngü oluşmaz (ağaç).
  - Seçilen kenarların ağırlıkları toplamı en küçüktür.
- İki temel algoritma vardır: Prim ve Kruskal.

Derste kullanılan graf (ekrandan, 02:11:30):

```
A-B 7   A-D 5   B-C 8   B-D 9   B-E 7   C-E 5
D-E 15  D-F 6   E-F 8   E-G 9   F-G 11
```

Prim Algoritması

- Prim açgözlü (greedy) bir algoritmadır.
- Açgözlü yaklaşım: her adımda yerel olarak en iyi seçenek seçilir.
- Bir düğümden başlanır. Ağaca her adımda bir düğüm eklenir.
- Eklenen düğüm, ağaçtaki düğümlerden çıkan en küçük ağırlıklı kenarla bağlanır. Döngü oluşturan kenar alınmaz.

A düğümünden başlayan çözüm:

Adım	Seçilen kenar	Ağırlık	Not
1	A–D	5	A'nın en küçük kenarı
2	D–F	6	A ve D'den çıkan en küçük kenar (7'den küçük)
3	A–B	7	
4	B–E	7	
5	E–C	5	E ağaca girince C'ye 5'lik kenar açıldı
6	E–G	9	E–F (8) ve B–C (8) döngü oluşturduğu için alınmadı. B–D (9) da alınmadı

- Toplam: $5 + 6 + 7 + 7 + 5 + 9 = 39$.
- 7 düğüm vardı. 6 kenar alındığında ağaç tamamlandı.

Kruskal Algoritması

- Önce tüm kenarlar ağırlıklarına göre küçükten büyüğe sıralanır.
- Sırayla en küçük kenar alınır. Döngü oluşturan kenar atlanır.
- n düğüm için $n - 1$ kenar alınınca durulur.

Sıralı kenarlar ve kararlar (ekrandan, 02:11:30):

Kenar	Ağırlık	Karar
A–D	5	Alındı
C–E	5	Alındı
D–F	6	Alındı
A–B	7	Alındı
B–E	7	Alındı (iki alt graf birleşti)
B–C	8	Alınmadı (döngü)
E–F	8	Alınmadı (döngü)
B–D	9	Alınmadı (döngü)
E–G	9	Alındı (6. kenar, iş bitti)
F–G	11	Bakılmadı
D–E	15	Bakılmadı

- Toplam yine 39'dur. Prim ile aynı ağaç elde edildi.

Kruskal İçin Etiket Dizisi Tasarımı

- Hangi düğümün hangi alt grafa ait olduğunu tutmak için bir etiket (label) dizisi kullanılır.
- Dizi indisleri düğümleri gösterir (A = 0, B = 1, …, G = 6). Başlangıçta tüm etiketler 0'dır. 0, düğümün henüz hiçbir alt grafa girmediği anlamına gelir.
- Kenar (u, v) için dört durum vardır:
  - **Durum 1:** `label[u] == 0` ve `label[v] == 0`. İkisi de boştadır. Yeni bir etiket numarası alınır (sayaç bir artırılır). İki düğüm bu etiketle işaretlenir. Yeni bir alt graf oluşur.
  - **Durum 2:** Etiketlerden yalnız biri 0. Sıfır olan düğüm, diğerinin etiketiyle işaretlenir. Düğüm o alt grafa katılır.
  - **Durum 3:** İki etiket de sıfırdan farklı ve birbirinden farklı. İki ayrı alt graf birleşir. Bir etiketi taşıyan tüm düğümler diğer etiketle yeniden işaretlenir.
  - **Durum 4:** `label[u] == label[v]`. İkisi aynı alt graftadır. Kenar alınırsa döngü oluşur. Kenar alınmaz.

Örnekteki etiketlerin değişimi:

```
                A  B  C  D  E  F  G
Başlangıç       0  0  0  0  0  0  0
A-D (durum 1)   1  0  0  1  0  0  0
C-E (durum 1)   1  0  2  1  2  0  0
D-F (durum 2)   1  0  2  1  2  1  0
A-B (durum 2)   1  1  2  1  2  1  0
B-E (durum 3)   1  1  1  1  1  1  0   (tüm 2'ler 1 oldu)
B-C (durum 4)   alınmadı
E-F (durum 4)   alınmadı
B-D (durum 4)   alınmadı
E-G (durum 2)   1  1  1  1  1  1  1
```

- Öğrenci önerisi: Döngü "alınan kenar sayısı, düğüm sayısından bir eksik olana kadar" sürsün. Hoca bunu onayladı.

Komşuluk Listesinin C ile Gerçekleştirilmesi

- Grafın yapısı: düğüm sayısı ve her düğüm için bir bağlı listenin başını tutan işaretçi dizisi.
- `adjLists` çift işaretçidir (`struct node**`). Dizinin her gözü bir bağlı liste gösterir.
- Liste düğümünde yalnız komşu düğümün numarası ve sonraki düğümün adresi vardır.

- Hoca önce ağırlığı da liste düğümüne yazdı, sonra geri aldı. Basit sürümde yalnız `vertex` ve `next` kalmalıdır. Ağırlık kenar eklenirken ayrıca ele alınmalıdır.

Slayttaki kod (ekrandan, 02:14:40 – 02:26:20):

```
struct node
{
    int vertex;
    struct node* next;
};
struct Graph
{
    int numVertices;
    struct node** adjLists;
};

struct node* createNode(int v)
{
    struct node* newNode = malloc(sizeof(struct node));
    newNode->vertex = v;
    newNode->next = NULL;
    return newNode;
}

struct Graph* createGraph(int vertices)
{
    int i;
    struct Graph* graph = malloc(sizeof(struct Graph));
    graph->numVertices = vertices;
    graph->adjLists = malloc(vertices * sizeof(struct node*));
    for (i = 0; i < vertices; i++)
        graph->adjLists[i] = NULL;
    return graph;
}

void addEdge(struct Graph* graph, int src, int dest)
{
    struct node* newNode = createNode(dest);
    newNode->next = graph->adjLists[src];
    graph->adjLists[src] = newNode;
    newNode = createNode(src);
    newNode->next = graph->adjLists[dest];
    graph->adjLists[dest] = newNode;
}
```

- `createGraph`: graf için yer ayrılır. Düğüm sayısı kadar liste başı ayrılır ve hepsi `NULL` yapılır. Başlangıçta hiçbir düğümün komşusu yoktur.
- `createNode`: yeni düğüm "havada" oluşturulur. `vertex` numarayı tutar, `next` `NULL`'dır.
- `addEdge`: önce hedef düğüm, kaynağın listesinin başına eklenir. Sonra kaynak düğüm, hedefin listesinin başına eklenir. Graf yönsüz olduğu için iki ekleme de gerekir.
- Başa ekleme örneği: 0'ın listesi `2 -> ...` iken 4 eklenirse önce `4->next` eski baş (2) yapılır. Sonra liste başı 4 olur.

Ders sonrası notlar

- Hoca, Prim'in başlangıç düğümüne bağlı olarak her zaman en küçük kapsayan ağacı vermeyebileceğini, Kruskal'ın ise global en iyi çözümü aradığını söyledi. Bu doğru değildir. Bağlı ve ağırlıklı bir grafta Prim hangi düğümden başlarsa başlasın bir en küçük kapsayan ağaç bulur. Kruskal da açgözlü bir algoritmadır ve o da her zaman en küçük kapsayan ağaç bulur. Eşit ağırlıklı kenarlar varsa iki algoritma farklı ağaçlar bulabilir, ama toplam ağırlık aynıdır.

- Hoca ağacı “yönsüz ve döngüsüz graf” olarak tanımladı. Tam tanımda ağaç ayrıca bağlıdır. Bağlı olmayan döngüsüz graf ormandır (forest).
- Listede “v ile w bağlı mı?” sorusunun en kötü durum maliyeti, toplam kenar sayısı değil, v’nin listesindeki eleman sayısıdır (derecesi).
- Kruskal’daki etiket dizisi, ayrık küme (disjoint set / union-find) yapısının basit bir hâlidir. Durum 3’te bir etiketin tüm düğümlerini değiştirmek her birleşmede $O(V)$ sürer.
- Slayttaki kodda `malloc` sonucu kontrol edilmiyor ve `malloc` için `<stdlib.h>` gerekir. Gerçek kodda `NULL` kontrolü yapılmalıdır.
- Altyazıda Whisper bazı terimleri bozmuştur: “pet” = path, “spelling tree” = spanning tree, “Kurzcal / Pruskal / Kuruskal” = Kruskal, “digri” = degree.

Not: İsterseniz bu dersin altyazı (.srt) dosyasını NotebookLM gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.